

Project Group: 21

April 6, 2025

Computational Photography Course Project: Depth from Monocular Image

by

Alan Zheng

Repository: https://github.com/JuicedCooky/cv_depth_perception

Abstract:

In this project, I aim to explore the process of extracting 3D depth maps from singular/monocular images. Since the architecture of the model used here is quite 'simple' with only 223,333 parameters. The architecture used in converting images to depth maps is a Residual Encoder-Decoder Network that uses convolutional layers in between. The data used to train the model is the Make-3D from Stanford University and Cornell University, which contains outdoor images with corresponding depth maps obtained from laser imaging.

1 Dataset

1.1 Make3D

This dataset contains only training and test datasets with 400 and 100+ images, respectively. See figure 1 for an example.

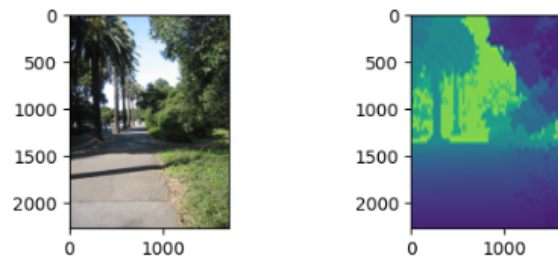


Figure 1: Image of Make3D dataset with corresponding depth map.

1.2 Data Manipulation

To better visualize and separate values that are from the Make3D dataset depth maps, I applied histogram equalization using OpenCV's `equalizeHist` function, which separates the values into a larger range (see figure 2).

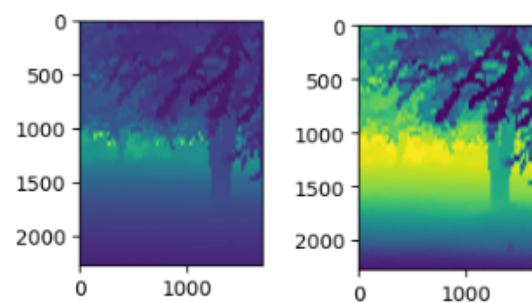


Figure 2: Application of histogram equalization of depth map

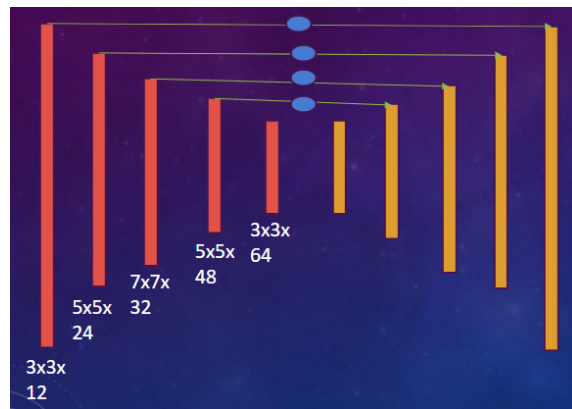


Figure 3: Visual of Encoder-Decoder Model

2 Model

2.1 Encoder and Decoder

The Encoder-Decoder neural network is a convolution network-based model; it takes an image and applies a set of convolutions with different sized kernels and is downsized in this case using kernels with strides.

2.2 Layers

There are a total of 5 layers in the Encoder and 5 layers in the Decoder.

2.2.1 Encoder

Each layer consists of a convolution with strides to reduce image dimension, followed by a batch normalization across all kernels for each channel, and finally LeakyReLU for the activation function.

Batch normalization is used since there are variances and different scales of the multiple channels that represent different features of the image, and it normalizes the values across each channel so that when the training step evaluates each feature channel, one does not dominate another due to differences in values and variances.

I also used LeakyReLU instead of the standard ReLU for the activation function,

since for this application of discerning depths from an Image, the need to keep all information of the image is valuable (ex., position, color gradient, brightness intensities). So LeakyReLU allows for values that are negative to be evaluated to a value based on the slope (0.01 in this case) instead of 0.

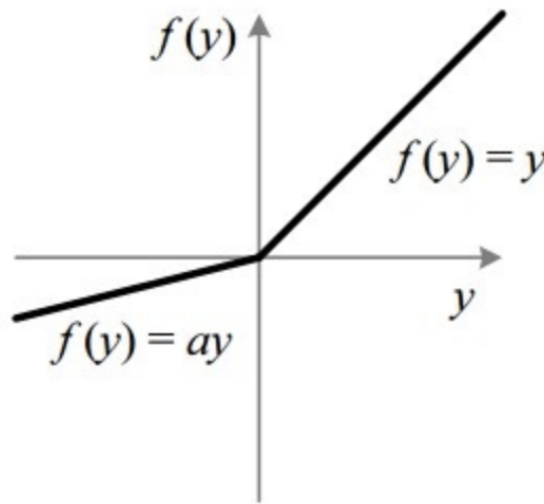


Figure 4: LeakyReLU activation function

2.2.2 Decoder

The Decoder section of the model is similar to the Encoder layers. The differences are that:

Instead of a convolution it applies a convolution transpose, which it creates features instead of detecting and extracting features. This is also then applied by pooling to ensure the data shape to be the same for the next step.

To preserve the structural and image features, each output data from the encoder layers are applied to the input of the decoder layers. There are parameters weights applied to the residual encoder layer output such that we don't have to manually tune the weights, and the training step will adapt the weights dynamically.

The final activation function uses a sigmoid in order to output a range of values between 0-1.

3 Loss

A hybrid loss approach was used in this model, the four losses were used to train the model:

- 1. L1 Loss (MAE)
 - The mean of the absolute difference of each pixel between the predicted and the ground truth.
- 2. Structural Similarity Loss
 - Common in comparing the differences between two images, it measures the structure and intensities between two images using variance and covariance.
- 3. Gradient Loss
 - The loss is computed between the gradients of the x-axis and the y-axis gradients. This helps separate the object edges by evaluating the gradient changes around an object.
- 4. SI Loss
 - Scale invariant loss takes the relative differences between the two images rather than just the actual intensity differences. Rather than computing how different the pixel values are, it takes relative values and ratios within its own image.

Each loss has an adaptive weight applied to it during training, which is important due to the varying magnitudes that are produced by these different methods of loss calculations.

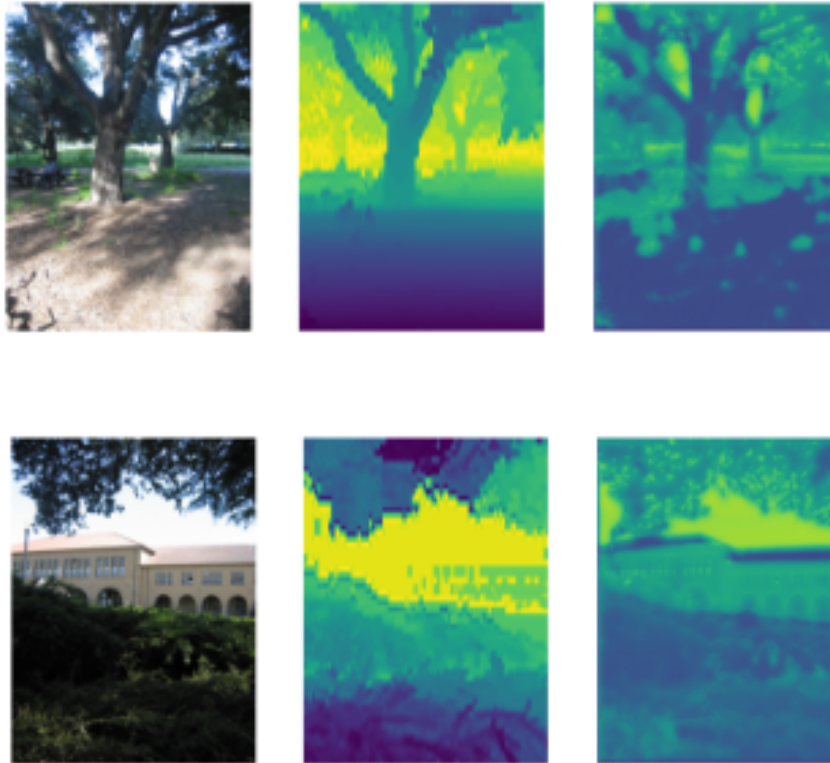
A sigmoid activation function is also applied to the weights such that the values are constrained between 0-1.

4 Results

The model can discern objects that are closer in the scene or objects that are not within the background and can show to separation of objects' general discrete distances, for example, close, mid, and far depths.

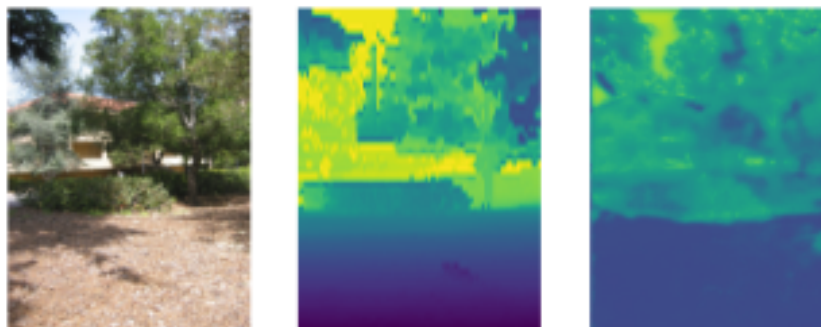
The left image is the input image, the middle image is the ground truth depth map, and the right image is the predicted depth map from the image.



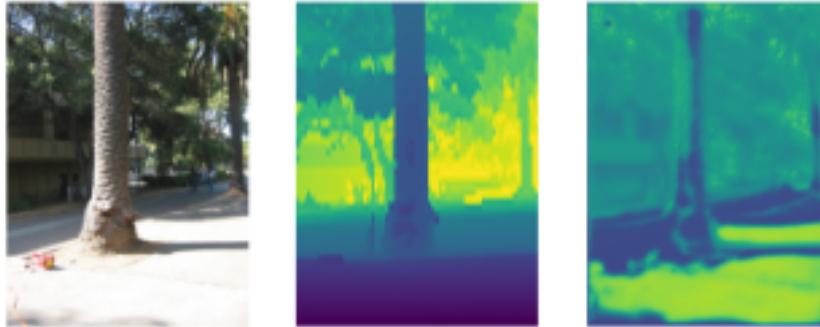


Here we see that there are some properties of the images that the model cannot handle.

For this first image we can see that the amount of clutter of trees in the mid-section of the image makes it harder for the model to further separate the objects within and tends to generalize the structure into one.



This example shows the effects/weight of brightness and shadows. We can see that if part of the objects contain shadows and bright areas, the model tends to separate the two as different depths. It is probably due to the fact that it uses dark areas and bright areas from a light source as features.



5 Conclusions

I described the application of my simple depth detection neural network model that extracts information from a single image. The architecture and the method of training the model was discussed, and some issues were presented in the resulting images.